

Coulomb Blockade Setup Package Description.

Version 0.0.4

January 22, 2025

1 Introduction

Coulomb blockade package (CBSetup.py, depends on simulator.py) is intended for direct simulation of single-electron transport in Coulomb systems. Such systems consist of Coulomb *islands* connected by *tunnel junctions* and optionally non-conducting *capacitances* to each other and the *leads*. The transport in the systems is affected by *gate electrodes*. Typical output computed consists of average electric currents in the terminals at fixed settings of the voltages at the leads and at the gate electrodes.

The actual use of the package starts with creating an instance of the **CB-Setup** class. Then the setup is *built* by adding the islands, terminals, capacitances and the junctions that connecting the islands to terminals and to each other. It is so that each element of the setup should be given a name, and addressed by this name. If you do not like giving names, make your own script to do this for you.

The build-up must be *finished* before the use of the setup. After this, there is no way to change the parameters of the setup or add/remove the elements except the voltages. Typical execution loop is to give a simulation run, change voltages, repeat the run, and so on, with the goal, for instance, to compute the $I - V$ characteristics of the setup. You should choose correct duration of the simulation run to achieve the desired accuracy.

A technical note: the simulator *cashes* the tunnelling rates for a given charging state of the setup. This accelerates the execution if the possible charging states are not that many. If you experience problems here, try to switch off the caching: `ClassInstance.cashing=False`.

2 Building

Import **CBSetup** from the package. The instance of the class, our setup, is created by calling **CBSetup()**. The following member functions of the instance add the elements of the setup without connecting them:

add_lead(string terminal_name)

Adds a lead with a given name.

add_island(string terminal_name)

Adds an island with a given name.

The following functions add and connect.

add_gate(string gate_name, string island_name, double capacitance)

Adds a gate electrode with a given name connecting it to an existing island with the provided capacitance. In the current version, a gate electrode controls a single island only.

add_junctionBTI (string junction_name, string firstisland, string secondisland, double conductance, double capacitance)

Adds a tunnel junction *between the islands* **firstisland**, **secondisland** with the provided conductance and capacitance.

add_junctionBIL (string junction_name, string island, string lead, double conductance, double capacitance)

Adds a tunnel junction *between an island* **island** and *a lead* **lead** with the provided conductance and capacitance.

add_capacitanceBTI (string capacitance_name, string firstisland, string secondisland, double capacitance)

Adds a non-conducting capacitance *between the islands* **firstisland**, **secondisland** with the provided capacitance.

add_capacitanceBIL (string junction_name, string island, string lead, double conductance, double capacitance)

Adds a non-conducting capacitance *between an island* **island** and *a lead* **lead** with the provided capacitance.

This finishes the build:

finish()

Please note that the current version does not check if all islands or leads are connected to anything. This is up to you to guarantee.

3 The setting functions

set_lead_voltage(string lead_name, double value)

Sets the voltage of the named lead to the value provided.

set_gate_voltage(string gate_name, value)

Sets the voltage of the named gate electrode to the value provided.

set_lead_voltages (numpy_array varray)

This is an alternative way of setting lead voltages: you provide the array of the

voltages of all leads. For this, it is useful to know the correspondence between the lead names and their places in the array. To this end, print `CBSetup.leaddic`.

set_gate_voltages (numpy_array varray)

This is an alternative way of setting lead voltages: you provide the array of the voltages of all leads. For this, it is useful to know the correspondence between the lead names and their places in the array. To this end, print `CBSetup.gatedic`.

An important global parameter of the setup is the *temperature*. Its default value is `CBSetup.temperature=0.1` and can be changed at every moment. Do not set it to zero or to a very small value since it may cause arithmetic interruptions.

4 Simulations

There are two functions available to simulate the single-electron transport in the setup built.

give_a_run (string stop='time', double timeinterval=1000.0, int maxsteps=1000, tuple charge = ("0",100),debug=False)

This runs a single simulation cycle. It is easy to start the cycle, it requires a thought where to *stop* it. In the present version, there are three ways to stop the cycle: possible values of the **stop** argument are 'time', 'steps'. If **stop** == 'time', the simulator stops after a certain simulated time is expired. This time is set by **timeinterval**. If **stop** == 'steps', the cycle stops after a certain number of tunneling events anywhere in the setup. This number is specified by **maxsteps**. If **stop** == 'charge', the cycle stops after an absolute value of charge transferred in a certain junction between a lead and an island exceeds a certain integer value. You need to specify **charge** = (string junction_name, int value).

The function returns the list of estimated currents in each lead. The currents are estimated in the most straightforward way: the number of charge transferred is divided by the time expired.

If you set **debug** == True, in the end of the cycle the program prints the CPU time expired, simulated time expired, number of events that took place, and the cache size.

Note 1: Mind the accuracy! The relative accuracy of the currents is inversely proportional to the square root of the simulation time. The best way to estimate the accuracy, and therefore the time required to achieve the desired one, is to make several runs at the same voltage settings and compare the results. Yet better control is achieved by using **stop** == 'charge', then the accuracy is defined by the value you set not depending on how many other events are happening and how fast or slow they occur.

Note 2: For curious minds, the simulator accumulates more information than just the currents returned. Right after the cycle, the dictionary `CBSetup.simulator.statetimes` contains times the setup has spent in each charging state during the cycle. The integer array `CBSetup.simulator.eventsoccured`

contains numbers of all possible tunnelling events that took place during the cycle.

runit(int numsteps, numpy_array initial_leads, numpy_array initial_gates, numpy_array final_leads, numpy_array final_gates, string stop='time', double timeinterval=1000.0, int maxsteps=1000, tuple charge = ("0",100), debug=False)

This runs **numsteps**+1 simulation cycles while the lead and gate voltages are changing from their initial vales specified in the arrays **initial_leads**, **initial_gates** to the final values specified in **final_leads**, **initial_gates**. It returns a list of the length **numsteps**+1 each item of which is the list of the currents.

The settings of **stop** and **debug** are applied to each cycle. In addition, if **debug** == True, the program prints the number of the run.

There are two functions to get more insight into the simulation.

sorted_cache (int num=5)

This function inspects cache available after a simulation cycle initiated with **give_it_a_run**(). It returns the list of the charge configurations that appear during the cycle along with their probabilities. The list is sorted, so the most probable states come on the top. The variable **num** controls how many configurations are returned. Set **num**=0 if you want all states.

simulator.movie_theater (string filename, int runs=25)

This function is useful if you want to understand the processes in the setup. It makes **runs** simulation steps and writes the results to the file **filename**. The output has the form configuration: time spend in this configuration. So it is like a movie script indicating a duration of changing scenes.

Check the example files *CBSetupProbe.py*, *CBSetupProbeDoubleChain.py*
Happy computing!